

SQL Part 7: CTEs (WITH), Recursive Queries & EXISTS Made Easy — Free Practice Guide

Beyond School · Ready to learn in a fun way! Anyone can learn — easy! Nothing to install, just a web browser.

Step 1: Open a free online SQL playground

- sqliteonline.com — choose "SQLite", paste, click Run.
- db-fiddle.com — pick "SQLite" as the database, paste and Run.
- sqlime.org — a simple SQLite editor in your browser.
- sqlbolt.com — free interactive lessons for extra practice.

Step 2: Paste this setup script and click Run

It creates the same library tables used in the video. If anything goes wrong, just run it again in a fresh playground.

```
CREATE TABLE books (
  id INTEGER PRIMARY KEY,
  title TEXT,
  author TEXT,
  year INTEGER,
  genre TEXT,
  pages INTEGER
);
INSERT INTO books VALUES
  (1, 'Harry Potter', 'J.K. Rowling', 1997, 'Fantasy', 309),
  (2, 'Matilda', 'Roald Dahl', 1988, 'Comedy', 240),
  (3, 'Wonder', 'R.J. Palacio', 2012, 'Drama', 310),
  (4, 'The Hobbit', 'J.R.R. Tolkien', 1937, 'Fantasy', 310),
  (5, 'Holes', 'Louis Sachar', 1998, 'Mystery', 233),
  (6, 'Charlotte''s Web', 'E.B. White', 1952, 'Classic', 192),
  (7, 'Dog Man', 'Dav Pilkey', 2016, 'Comedy', 240),
  (8, 'Coraline', 'Neil Gaiman', 2002, 'Fantasy', 162);

CREATE TABLE members (
  member_id INTEGER PRIMARY KEY,
  name TEXT,
  city TEXT,
  joined TEXT,
```

```
  email TEXT
);
INSERT INTO members VALUES
  (1, 'Aisha', 'Austin', '2023-01-15', 'aisha@mail.com'),
  (2, 'Ben', 'Denver', '2023-03-02', NULL),
  (3, 'Carlos', 'Austin', '2024-06-20', 'carlos@mail.com'),
  (4, 'Dana', 'Boston', '2024-09-10', NULL),
  (5, 'Eli', 'Denver', '2025-02-01', 'eli@mail.com');

CREATE TABLE loans (
  loan_id INTEGER PRIMARY KEY,
  book_id INTEGER,
  member_id INTEGER,
  loan_date TEXT,
  return_date TEXT
);
INSERT INTO loans VALUES
  (1, 1, 1, '2026-01-05', '2026-01-19'),
  (2, 3, 1, '2026-02-10', NULL),
  (3, 2, 2, '2026-01-20', '2026-02-03'),
  (4, 4, 3, '2026-03-01', '2026-03-15'),
  (5, 1, 3, '2026-04-02', NULL),
  (6, 7, 4, '2026-04-10', '2026-04-24'),
  (7, 7, 2, '2026-05-01', NULL),
  (8, 5, 4, '2026-05-15', NULL);
```

Step 3: Run every query from the video, in order

Paste one at a time, click Run, and compare with the expected result.

Your first CTE

```
WITH long_books AS (    SELECT title, pages FROM books    WHERE pages >= 300 ) SELECT * FROM long_books ORDER BY title;
```

```
Expected result:
title | pages
Harry Potter | 309
The Hobbit | 310
Wonder | 310
```

CTE + JOIN

```
WITH reading AS (   SELECT l.member_id, SUM(b.pages) AS pages_read   FROM loans AS l JOIN books AS b ON l.book_id = b.id   GROUP BY l.member_id ) SELECT m.name, r.pages_read FROM reading AS r JOIN members AS m ON r.member_id = m.member_id ORDER BY r.pages_read DESC, m.name;
```

```
Expected result:
name | pages_read
Aisha | 619
Carlos | 619
Ben | 480
Dana | 473
```

Two CTEs together

```
WITH reading AS (   SELECT l.member_id, SUM(b.pages) AS pages_read   FROM loans AS l JOIN books AS b ON l.book_id = b.id    GROUP BY l.member_id), avg_reading AS (    SELECT AVG(pages_read) AS avg_pages  FROM  reading)  SELECT  r.member_id,  r.pages_read,  a.avg_pages  FROM  reading  AS  r,  avg_reading AS a WHERE r.pages_read > a.avg_pages;
```

```
Expected result:
member_id | pages_read | avg_pages
1 | 619 | 547.75
3 | 619 | 547.75
```

Recursive: count 1 to 10

```
WITH RECURSIVE numbers(x) AS (   SELECT 1   UNION ALL   SELECT x + 1 FROM numbers WHERE x < 10 ) SELECT x FROM numbers;
```

```
Expected result:
x
1
2
3
4
5
6
7
8
9
10
```

Recursive: a list of months

```
WITH RECURSIVE months(m) AS (   SELECT '2026-01-01'   UNION ALL   SELECT date(m, '+1 month') FROM months    WHERE m < '2026-05-01') SELECT m AS month,    (SELECT COUNT(*) FROM loans     WHERE strftime('%m', loan_date) = strftime('%m', m)) AS loans FROM months;
```

```
Expected result:
month | loans
2026-01-01 | 2
2026-02-01 | 1
2026-03-01 | 1
2026-04-01 | 2
2026-05-01 | 2
```

Longer than its genre average

```
SELECT title, genre, pages FROM books AS b WHERE pages > (   SELECT AVG(pages) FROM books AS b2   WHERE b2.genre = b.genre);
```

```
Expected result:
title | genre | pages
Harry Potter | Fantasy | 309
The Hobbit | Fantasy | 310
```

Correlated subquery in SELECT

```
SELECT b.title,   (SELECT COUNT(*) FROM loans AS l    WHERE l.book_id = b.id) AS times FROM books AS b ORDER BY times DESC, b.title;
```

```
Expected result:
title | times
Dog Man | 2
Harry Potter | 2
Holes | 1
Matilda | 1
The Hobbit | 1
Wonder | 1
Charlotte's Web | 0
Coraline | 0
```

EXISTS

```
SELECT m.name FROM members AS m WHERE EXISTS (   SELECT 1 FROM loans AS l   JOIN books AS b ON l.book_id = b.id   WHERE l.member_id = m.member_id     AND b.genre = 'Fantasy');
```

```
Expected result:
name
Aisha
Carlos
```

NOT EXISTS

```
SELECT title FROM books AS b WHERE NOT EXISTS (   SELECT 1 FROM loans AS l   WHERE l.book_id = b.id);
```

```
Expected result:
title
Charlotte's Web
Coraline
```

NOT EXISTS for members

```
SELECT name FROM members AS m WHERE NOT EXISTS (   SELECT 1 FROM loans AS l   WHERE l.member_id = m.member_id);
```

```
Expected result:
name
Eli
```

Earliest member in each city

```
SELECT name, city, joined FROM members AS m WHERE joined = (   SELECT MIN(joined) FROM members AS m2   WHERE m2.city = m.city);
```

```
Expected result:
name | city | joined
Aisha | Austin | 2023-01-15
Ben | Denver | 2023-03-02
Dana | Boston | 2024-09-10
```

Step 4: Challenges (try first!)

C1. Use EXISTS to list books that are still out on loan.

C2. Use a recursive CTE to count down from 5 to 1.

C3. Use a CTE to find genres with more than one book, then list those books.

Answers

C1. Use EXISTS to list books that are still out on loan.

```
SELECT title FROM books AS b
WHERE EXISTS (SELECT 1 FROM loans AS l WHERE l.book_id = b.id AND l.return_date IS NULL);
```

```
Expected result:
title
Harry Potter
Wonder
Holes
Dog Man
```

C2. Use a recursive CTE to count down from 5 to 1.

```
WITH RECURSIVE countdown(n) AS (
  SELECT 5 UNION ALL SELECT n - 1 FROM countdown WHERE n > 1)
SELECT n FROM countdown;
```

```
Expected result:
n
5
4
3
2
1
```

C3. Use a CTE to find genres with more than one book, then list those books.

```
WITH big AS (SELECT genre FROM books GROUP BY genre HAVING COUNT(*) > 1)
SELECT title, genre FROM books WHERE genre IN (SELECT genre FROM big);
```

```
Expected result:
title | genre
Harry Potter | Fantasy
Matilda | Comedy
The Hobbit | Fantasy
Dog Man | Comedy
Coraline | Fantasy
```

Keep going!

Watch the whole free SQL course on Beyond School, starting with SQL Part 1: https://youtu.be/72JvYu2OtiA
Beyond School — Ready to learn in a fun way!